

Finite Difference Method for Elliptic Equations

Julio A. Medina
Universidad de San Carlos
School of Physical and Mathematical Sciences
Master's Program in Physics
julioantonio.medina@gmail.com

1 Elliptic partial differential equations

The elliptic partial differential equation considered in this work is Poisson's equation

$$\nabla^2 u(x, y) \equiv \frac{\partial^2 u}{\partial x^2}(x, y) + \frac{\partial^2 u}{\partial y^2}(x, y) = f(x, y) \quad (1)$$

in $R = \{(x, y) \mid a < x < b, c < y < d\}$, with $u(x, y) = g(x, y) \in S$, where S denotes the boundary of R . If f and g are continuous on their domains, then the equation has a unique solution.

1.1 Choosing a mesh

The method used here is a two-dimensional adaptation of the finite difference method for problems with linear boundaries, as discussed in [1]. The first step is to choose integers n and m in order to define the step sizes $h = (b - a)/n$ and $k = (d - c)/m$. In this way, the interval $[a, b]$ is partitioned into n equal parts of width h , and the interval $[c, d]$ is partitioned into m equal parts of width k , forming a mesh or grid as shown in the figure.

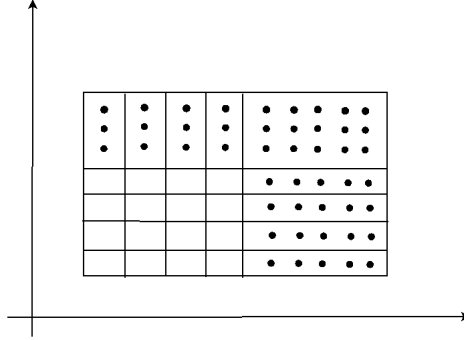


Figure 1: $n \times m$ grid

This mesh is formally constructed by drawing vertical and horizontal lines over the rectangle R at the points with coordinates (x_i, y_j) , where

$$x_i = a + ih, \text{ for each } i = 0, 1, 2, \dots, n \text{ and } y_j = c + jk, \text{ for each } j = 0, 1, 2, \dots, m. \quad (2)$$

The lines corresponding to $x = x_i$ and $y = y_j$ form the grid, and their intersections are the mesh points. For each interior mesh point (x_i, y_j) , with $i = 1, 2, \dots, n-1$ and $j = 1, 2, \dots, m-1$, a Taylor series in the variable x around the point x_i can be used to generate the centered-difference formula

$$\frac{\partial^2 u}{\partial x^2}(x_i, y_j) = \frac{u(x_{i+1}, y_j) - 2u(x_i, y_j) + u(x_{i-1}, y_j)}{h^2} - \frac{h^2}{12} \frac{\partial^4 u}{\partial x^4}(\xi_i, y_j) \quad (3)$$

where $\xi_i \in (x_{i-1}, x_{i+1})$. Similarly, using the Taylor series in the variable y around the point y_j , one obtains the centered difference

$$\frac{\partial^2 u}{\partial y^2}(x_i, y_j) = \frac{u(x_i, y_{j+1}) - 2u(x_i, y_j) + u(x_i, y_{j-1}))}{k^2} - \frac{k^2}{12} \frac{\partial^4 u}{\partial y^4}(x_i, \eta_j) \quad (4)$$

where $\eta_j \in (y_{j-1}, y_{j+1})$. Substituting these formulas into equation 1 allows Poisson's equation to be written at the points (x_i, y_j) as

$$\begin{aligned} & \frac{u(x_{i+1}, y_j) - 2u(x_i, y_j) + u(x_{i-1}, y_j)}{h^2} + \frac{u(x_i, y_{j+1}) - 2u(x_i, y_j) + u(x_i, y_{j-1}))}{k^2} \\ &= f(x_i, y_j) + \frac{h^2}{12} \frac{\partial^4 u}{\partial x^4}(\xi_i, y_j) + \frac{k^2}{12} \frac{\partial^4 u}{\partial y^4}(x_i, \eta_j) \end{aligned} \quad (5)$$

for each $i = 1, 2, \dots, n-1$ and $j = 1, 2, \dots, m-1$. The boundary conditions are

$$\begin{aligned} & u(x_0, y_j) = g(x_0, y_j) \quad \text{and} \quad u(x_n, y_j) = g(x_n, y_j), \quad \text{for each } j = 0, 1, \dots, m; \\ & u(x_i, y_0) = g(x_i, y_0) \quad \text{and} \quad u(x_i, y_m) = g(x_i, y_m), \quad \text{for each } i = 1, 2, \dots, n-1. \end{aligned}$$

1.2 Finite Difference Method

In difference-equation form, the finite difference method is

$$2 \left[\left(\frac{h}{k} \right)^2 + 1 \right] w_{ij} - (w_{i+1,j} + w_{i-1,j}) - \left(\frac{h}{k} \right)^2 (w_{i,j+1} + w_{i,j-1}) = -h^2 f(x_i, y_j) \quad (6)$$

for each $i = 1, 2, \dots, n-1$ and $j = 1, 2, \dots, m-1$, where

$$\begin{aligned} & w_{0j} = g(x_0, y_j) \quad \text{and} \quad w_{nj} = g(x_n, y_j), \quad \text{for each } j = 0, 1, \dots, m; \\ & w_{i0} = g(x_i, y_0) \quad \text{and} \quad w_{im} = g(x_i, y_m), \quad \text{for each } i = 1, 2, \dots, n-1. \end{aligned} \quad (7)$$

Here, w_{ij} approximates $u(x_i, y_j)$. This method has a truncation error of order $O(h^2 + k^2)$. Equation 6 involves approximations to $u(x, y)$ at the points

$$(x_{i-1}, y_j), (x_{i+1}, y_j), (x_i, y_j), (x_i, y_{j-1}), \text{ and } (x_i, y_{j+1}).$$

Using the information provided by the boundary conditions 7 whenever the system defined by 6 allows it, that is, at the points adjacent to the boundary defined by the previously constructed grid (1.1), one obtains a system of equations with a coefficient matrix of dimension $(n-1)(m-1) \times (n-1)(m-1)$. The unknowns are the approximations w_{ij} to $u(x_i, y_j)$ at the interior mesh points (1.1).

The linear system for the unknowns w_{ij} can be represented more efficiently for matrix operations by relabeling the interior mesh points. A recommended relabeling scheme can be found in [2] and is defined as follows:

$$P_l = (x_i, y_j), \text{ and } w_l = w_{ij}, \quad (8)$$

where

$$l(i, j) = i + (m - 1 - j)(n - 1) \quad (9)$$

for each $i = 1, 2, \dots, n - 1$ and $j = 1, 2, \dots, m - 1$. With this scheme, the interior mesh points are labeled from left to right and from top to bottom. This ensures that the resulting linear system for finding w_{ij} has a banded matrix with maximum bandwidth $2n - 1$, specifically a block-tridiagonal matrix. This technique was used to implement the algorithm in Python; see algorithms 1 and 2.

2 Finite Difference Method Algorithm for Poisson's Equation

To obtain an algorithm that generates and solves the finite difference method for the elliptic partial differential equations described in the previous section, the relabeling scheme introduced above is used. This ensures that a block-tridiagonal system is obtained. For these block-tridiagonal systems, it is convenient to apply a generalization of the Crout factorization algorithm [2] for block-tridiagonal matrices.

Another iterative algorithm that can be used is the SOR relaxation method. With this in mind, a strategy for constructing a finite difference algorithm can be outlined as follows:

- Construct the system associated with the approximations w_{ij} to $u(x_i, y_j)$ using the relabeling scheme presented above.
- Apply a method for solving linear systems: the generalized Crout factorization or the iterative SOR method.
- Solve for w_{ij} .

2.1 Finite difference algorithm: linear system

2.1.1 Relabeling function l

The function l is implemented to relabel the linear system of equations.

Algorithm 1 Function L for relabeling the variables of the linear system

```

function L( $i, j, n, m$ )
    return  $(i + 1) + (m - 1 - (j + 1)) \times (n - 1) - 1$ 
end function

```

2.1.2 Associated linear system

With this function, the system associated with the finite difference method is created.

Algorithm 2 Finite Difference Linear System

```

function FINITE_DIFFERENCE_LINEAR_SYSTEM( $a, b, c, d, n, m, f, g$ )
     $h \leftarrow (b - a)/n$ 
     $k \leftarrow (d - c)/m$ 
     $x \leftarrow \text{numpy.linspace}(a, b, n + 1)$ 
     $y \leftarrow \text{numpy.linspace}(c, d, m + 1)$ 
     $W_{ij} \leftarrow \text{zeros}(((n - 1) \times (m - 1)), ((n - 1) \times (m - 1)))$ 
     $w \leftarrow \text{zeros}((n - 1) \times (m - 1))$ 
    for  $i \leftarrow 0$  to  $n - 2$  do
        for  $j \leftarrow 0$  to  $m - 2$  do
             $W_{ij}[L(i, j, n, m), L(i, j, n, m)] \leftarrow 2((h/k)^2 + 1)$ 
            if  $i \neq 0$  and  $i \neq n - 2$  then
                 $W_{ij}[L(i, j, n, m), L(i + 1, j, n, m)] \leftarrow -1$ 
                 $W_{ij}[L(i, j, n, m), L(i - 1, j, n, m)] \leftarrow -1$ 
            else
                if  $i = 0$  then
                     $W_{ij}[L(i, j, n, m), L(i + 1, j, n, m)] \leftarrow -1$ 
                     $w[l(i, j, n, m)] \leftarrow g(x[i], y[j + 1], a, b, c, d)$ 
                end if
                if  $i = n - 2$  then
                     $W_{ij}[L(i, j, n, m), L(i - 1, j, n, m)] \leftarrow -1$ 
                     $w[L(i, j, n, m)] \leftarrow g(x[i + 2], y[j + 1], a, b, c, d)$ 
                end if
            end if
            if  $j \neq 0$  and  $j \neq m - 2$  then
                 $W_{ij}[L(i, j, n, m), L(i, j + 1, n, m)] \leftarrow -(h/k)^2$ 
                 $W_{ij}[L(i, j, n, m), L(i, j - 1, n, m)] \leftarrow -(h/k)^2$ 
            else
                if  $j = 0$  then
                     $W_{ij}[L(i, j, n, m), L(i, j + 1, n, m)] \leftarrow -(h/k)^2$ 
                     $aux \leftarrow (h/k)^2 * g(x[i + 1], y[j], a, b, c, d)$ 
                     $w[L(i, j, n, m)] \leftarrow w[L(i, j, n, m)] + aux$ 
                end if
                if  $j = m - 2$  then
                     $W_{ij}[L(i, j, n, m), L(i, j - 1, n, m)] \leftarrow -(h/k)^2$ 
                     $aux \leftarrow (h/k)^2 * g(x[i + 1], y[j + 2], a, b, c, d)$ 
                     $w[L(i, j, n, m)] \leftarrow w[L(i, j, n, m)] + aux$ 
                end if
            end if
             $w[L(i, j, n, m)] \leftarrow -h^2 f(x[i + 1], y[j + 1]) + w[L(i, j, n, m)]$ 
        end for
    end for
    return  $W_{ij}, w$ 
end function

```

2.2 Generalized Crout factorization algorithm for block-tridiagonal matrices

This is the implementation of the generalized Crout factorization algorithm for block-tridiagonal matrices. The standard Crout factorization algorithm is valid for tridiagonal matrices, but not directly for block-tridiagonal matrices.

Algorithm 3 Crout Generalization Algorithm for Tridiagonal Block Matrices

Require: $A \in \mathbb{R}^{N \times N}$, $K \in \mathbb{R}^N$, $n \in \mathbb{N}$

Ensure: $sol \in \mathbb{R}^N$

```

function CROUT_GENERALIZATION( $A, K, n$ )
   $N \leftarrow \lfloor \frac{\text{shape}(A)[1]}{n} \rfloor$ 
   $W \leftarrow []$ 
   $B_1 \leftarrow A_{1:n, 1:n}$ 
   $C_1 \leftarrow A_{1:n, n:n+n}$ 
   $W.append(\text{inv}(B_1) \cdot C_1)$ 
  for  $i \leftarrow 2$  to  $N - 1$  do
     $B_i \leftarrow A_{(i-1)n:in, (i-1)n:in}$ 
     $A_i \leftarrow A_{(i-1)n:in, (i-2)n:(i-1)n}$ 
     $C_i \leftarrow A_{(i-1)n:in, in+1:n+(i-1)n}$ 
     $W.append(\text{inv}(B_i - A_i \cdot W_{i-2}) \cdot C_i)$ 
  end for
   $B_N \leftarrow A_{1:n, 1:n}$ 
   $K_1 \leftarrow K_{1:n}$ 
   $G_1 \leftarrow \text{inv}(B_1) \cdot K_1$ 
   $G \leftarrow []$ 
   $G.append(G_1)$ 
  for  $i \leftarrow 2$  to  $N$  do
     $K_i \leftarrow K_{(i-1)n:in}$ 
     $B_i \leftarrow A_{(i-1)n:in, (i-1)n:in}$ 
     $A_i \leftarrow A_{(i-1)n:in, (i-2)n:(i-1)n}$ 
     $G.append(\text{inv}(B_i - A_i \cdot W_{i-2}) \cdot (K_i - A_i \cdot G_{i-2}))$ 
  end for
   $Z \leftarrow G[:]$ 
  for  $i \leftarrow N - 2$  to  $0$  step  $-1$  do
     $Z_i \leftarrow G_i - W_i \cdot Z_{i+1}$ 
  end for
   $sol \leftarrow \text{concatenate}(Z)$ 
  return  $sol$ 
end function

```

2.3 Iterative SOR method for solving linear systems

Algorithm 4 Iterative SOR method for solving linear systems

Require: A is an $n \times n$ matrix, b is a right-hand-side vector of dimension n , $x^{(0)}$ is an initial proposed solution, ω is the relaxation parameter, ϵ is the tolerance, and max_{iter} is the maximum number of iterations.

Ensure: x is an approximate solution of $Ax = b$.

```

function SOR( $A, b, \omega, \epsilon, max_{iter}, x^{(0)}$ )
     $k \leftarrow 1$ 
     $xo \leftarrow x^{(0)}$ 
     $tolerance \leftarrow \mathbf{True}$ 
    while  $k < max_{iter}$  and  $tolerance$  do
        for  $i = 1$  to  $n$  do
             $s \leftarrow \sum_{j=1}^{i-1} a_{ij}x_j + \sum_{j=i+1}^n a_{ij}xo_j$ 
             $x_i \leftarrow (1 - \omega)xo_i + \frac{\omega}{a_{ii}}(b_i - s)$ 
        end for
        if  $\|x - xo\| < \epsilon$  then
             $tolerance \leftarrow \mathbf{False}$ 
        end if
         $k \leftarrow k + 1$ 
         $xo = x$ 
    end while
    if  $k = max_{iter}$  then
        print "The maximum number of iterations has been reached"
    end if
    return  $x$ 
end function

```

2.4 Finite differences using generalized Crout factorization

Algorithm 5 Finite difference algorithm using generalized Crout factorization

Given a problem with boundary conditions of the form 1, define

$$a, b, c, d, n, m, f(x, y), g(x, y)$$

using the functions defined in algorithms 2 and 3.

```

 $A, b \leftarrow \text{finite\_difference\_linear\_system}(a, b, c, d, n, m, f, g)$ 
 $solution \leftarrow \text{Crout\_generalization}(A, b, n-1)$ 

```

This algorithm was implemented in Python using the NumPy library. The repository can be found at https://github.com/Julio-Medina/Finite_Difference_Method/tree/main/code.

2.5 Finite differences using the SOR relaxation method

Algorithm 6 Finite difference algorithm using the SOR relaxation method

Given a problem with boundary conditions of the form 1, define

$$a, b, c, d, n, m, f(x, y), g(x, y), \omega, \epsilon, x^{(0)}$$

using the functions defined in algorithms 2 and 4.

$A, b \leftarrow \text{finite_difference_linear_system}(a, b, c, d, n, m, f, g)$
 $\text{solution} \leftarrow \text{SOR}(A, b, \omega, \epsilon, \max_{iter}, x^{(0)})$

2.6 Examples

There is no better way to illustrate how the algorithm works than by solving some boundary-value problems.

2.6.1 Example 1

Solve the following problem:

$$\frac{\partial^2 u}{\partial x^2}(x, y) + \frac{\partial^2 u}{\partial y^2}(x, y) = 0 \quad (10)$$

for each (x, y) in the set $R = \{(x, y) | 0 < x < 0.5, 0 < y < 0.5\}$. The boundary conditions are

$$u(0, y) = 0, u(x, 0) = 0, u(x, 0.5) = 200x, u(0.5, y) = 200y.$$

If $n = m = 4$, the finite difference equation 6 gives

$$4w_{i,j} - w_{i+1,j} - w_{i-1,j} - w_{i,j-1} - w_{i,j+1} = 0 \quad (11)$$

for each $i = 1, 2, 3$ and $j = 1, 2, 3$.

For example, when $i = 1, j = 1$, one obtains

$$4w_{1,1} - w_{2,1} - w_{0,1} - w_{1,0} - w_{1,2} = 0. \quad (12)$$

Since $w_{1,0} = w_{0,1} = 0$, using the relabeling function 9 gives

$$l(1, 1) = 7, l(2, 1) = 8, l(1, 2) = 4.$$

Substituting into 12 and using the boundary conditions gives the equation

$$4w_7 - w_8 - w_4 = 0.$$

This process is repeated for the remaining indices, producing 9 linear equations for the variables $w_1, w_2, w_3, w_4, w_5, w_6, w_7, w_8, w_9$. The coefficient matrix of the

linear system obtained by iterating through all indices is

$$A = \begin{bmatrix} 4.0 & -1.0 & 0.0 & -1.0 & 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ -1.0 & 4.0 & -1.0 & 0.0 & -1.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & -1.0 & 4.0 & 0.0 & 0.0 & -1.0 & 0.0 & 0.0 & 0.0 \\ -1.0 & 0.0 & 0.0 & 4.0 & -1.0 & 0.0 & -1.0 & 0.0 & 0.0 \\ 0.0 & -1.0 & 0.0 & -1.0 & 4.0 & -1.0 & 0.0 & -1.0 & 0.0 \\ 0.0 & 0.0 & -1.0 & 0.0 & -1.0 & 4.0 & 0.0 & 0.0 & -1.0 \\ 0.0 & 0.0 & 0.0 & -1.0 & 0.0 & 0.0 & 4.0 & -1.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & -1.0 & 0.0 & -1.0 & 4.0 & -1.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 & -1.0 & 0.0 & -1.0 & 4.0 \end{bmatrix} \quad (13)$$

and the constant vector is

$$b = \begin{bmatrix} 25.0 \\ 50.0 \\ 150.0 \\ 0.0 \\ 0.0 \\ 50.0 \\ 0.0 \\ 0.0 \\ 25.0 \end{bmatrix}.$$

It is important to note that the resulting matrix 13 is a block-tridiagonal matrix, with each block having dimension 3×3 . Thus, the system

$$Aw = b \quad (14)$$

can be solved, giving

$$\begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ w_4 \\ w_5 \\ w_6 \\ w_7 \\ w_8 \\ w_9 \end{bmatrix} = \begin{bmatrix} 18.75 \\ 37.5 \\ 56.25 \\ 12.5 \\ 25 \\ 37.5 \\ 6.25 \\ 12.5 \\ 18.75 \end{bmatrix}. \quad (15)$$

2.6.2 Example 2

Solve the following problem:

$$\frac{\partial^2 u}{\partial x^2}(x, y) + \frac{\partial^2 u}{\partial y^2}(x, y) = 4 \quad (16)$$

for each (x, y) in the set $R = \{(x, y) | 0 < x < 1, 0 < y < 2\}$. The boundary conditions are

$$u(1, y) = \ln(y^2 + 1), u(x, 0) = 2 \ln x, u(x, 1) = \ln(x^2 + 1), u(2, y) = \ln(y^2 + 4).$$

If $h = k = \frac{1}{3}$, then $n = 3$, $m = 3$, and the finite difference equation 6 gives

$$4w_{i,j} - w_{i+1,j} - w_{i-1,j} - w_{i,j-1} - w_{i,j+1} = 4. \quad (17)$$

The associated coefficient matrix A is

$$\begin{bmatrix} 4.0 & -1.0 & -1.0 & 0.0 \\ -1.0 & 4.0 & 0.0 & -1.0 \\ -1.0 & 0.0 & 4.0 & -1.0 \\ 0.0 & -1.0 & -1.0 & 4.0 \end{bmatrix} \quad (18)$$

and the constant vector is

$$b = \begin{bmatrix} 1.165 \\ 2.774 \\ -0.444 \\ 1.165 \end{bmatrix}.$$

It is important to note that the resulting matrix 18 is a block-tridiagonal matrix, with each block having dimension 2×2 . Thus, the system

$$Aw = b \quad (19)$$

can be solved, giving

$$\begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ w_4 \end{bmatrix} = \begin{bmatrix} 0.5825 \\ 0.9849 \\ 0.1801 \\ 0.5825 \end{bmatrix}. \quad (20)$$

2.6.3 Example 3

Solve the following problem:

$$\frac{\partial^2 u}{\partial x^2}(x, y) + \frac{\partial^2 u}{\partial y^2}(x, y) = xe^y \quad (21)$$

for each (x, y) in the set $R = \{(x, y) | 0 < x < 2, 0 < y < 1\}$. The boundary conditions are

$$u(0, y) = 0, u(x, 0) = x, u(x, 1) = ex, u(2, y) = 2e^y.$$

With $n = 6$, $m = 5$, compare the approximations with the analytical solution $u(x, y) = xe^y$.

Using the functions created in Python gives the following table. This table can be found in the GitHub repository mentioned above under the name `error_table.csv`.

	i	j	x_i	y_j	w_{ij}	$u(x_i, y_j)$	$ u(x_i, y_j) - w_{ij} $
0	1	1	0.333333	0.2	0.407265	0.407134	0.000130
1	1	2	0.333333	0.4	0.497483	0.497275	0.000208
2	1	3	0.333333	0.6	0.607596	0.607373	0.000223
3	1	4	0.333333	0.8	0.742007	0.741847	0.000160
4	2	1	0.666667	0.2	0.814524	0.814269	0.000255
5	2	2	0.666667	0.4	0.994958	0.994550	0.000408
6	2	3	0.666667	0.6	1.215183	1.214746	0.000437
7	2	4	0.666667	0.8	1.484009	1.483694	0.000315
8	3	1	1.000000	0.2	1.221766	1.221403	0.000363
9	3	2	1.000000	0.4	1.492405	1.491825	0.000580
10	3	3	1.000000	0.6	1.822743	1.822119	0.000624
11	3	4	1.000000	0.8	2.225993	2.225541	0.000452
12	4	1	1.333333	0.2	1.628964	1.628537	0.000427
13	4	2	1.333333	0.4	1.989778	1.989100	0.000679
14	4	3	1.333333	0.6	2.430227	2.429492	0.000735
15	4	4	1.333333	0.8	2.967928	2.967388	0.000540
16	5	1	1.666667	0.2	2.036042	2.035671	0.000371
17	5	2	1.666667	0.4	2.486958	2.486374	0.000584
18	5	3	1.666667	0.6	3.037506	3.036865	0.000641
19	5	4	1.666667	0.8	3.709724	3.709235	0.000489

References

- [1] Richard L. Burden, J. Douglas Faires *Numerical Analysis*, (Ninth Edition). Brooks/Cole, Cengage Learning. 978-0-538-73351-9
- [2] Richard S. Varga. *Matrix Iterative Analysis*. Second Edition. Springer. DOI 10.1007/978-3-642-05156-2